CS114: Fundamentals of Programming

Class: BECE2018

Lab 09: Functions – I

Instructor: Qurrat-ul-ain Babar

Lab 09: Functions – I

Deliverables

1. Perform all **Tasks and Examples**. Write the code and take screen shots of the output of each task and share both in a word document.

Upload single word document on LMS. Follow the naming convention for file. No late submissions will be accepted.

Deadline: Check submission deadline on LMS.

The aim of this lab is to introduce basics of Functions which are most important aspect of C/C++ languages. In the next lab some other topics related to Functions will be covered.

Our aims today are:

- Function Introduction
- Function Syntax, Examples

Functions

Functions let you chop up a long program into named sections so that the sections can be reused throughout the program. Function has a name identifier, accepts parameters and returns a result. C++ functions can accept an unlimited number of parameters. ***In general, C++ does not care in what order you put your functions in the program, so long as the function name is known to the compiler before it is called.***

In this lab we would learn how to make our own functions which can perform certain tasks. Function is differentiated from a simple variable, by the parenthesis just after the name as main(). It just indicates to the compiler that it's a function.

The most common function that you have encountered is main() which you have been implementing throughout the lab. main() is the function whose name identifier is main with no parameters passed to it. Anything inside brackets are the parameters passed to the function. In case of multiple parameters, each of them are separated with a comma(,) in between. Parameters are written as the comma separated list within brackets just after the name of function.

Function is written as:

return_type function_name (comma separated list of parameters);

For example:

i. int main();

ii. float divide(int a, int b);

iii. void square(float x);

Here names of the functions are main, divide and square which take 0, 2 and 1 parameters respectively. These functions would return an integer, float and void (nothing) respectively.

The prototype of a function is a way of describing the parameters (also called as arguments of function) and parameter types with which a legal call to the function can be made. It contains the name of the function, its parameters and their type, and the return value.

Pattern to write the function prototype is shown below:

return_type function_name (comma separated list of parameters);

Three examples shown above are examples of the function prototype. If you are asked to write the prototype of any function it can be written as in the examples above.

As seen in the lecture, let's do an example which calls a function which prints ten asterisks (*) in line. (**********)

In this example main function just calls the function named asterisks which just prints the line of ten asterisks.

Example 1

```
#include <iostream>
using namespace std;

void asterisks(); // function prototype

int main ()
{
    asterisks(); // function call
}

void asterisks()  // function definition
{
    for (int i=0; i<10; i++)
        cout << "*";
}
```

Example 2

asterisks() function in the previous example does not take any argument and does not return anything represented by empty braces() after function name and void before function name respectively. After main the function definition of the function asterisks is given. Let's go one step ahead, function asterisks (int num) with a single argument.

```
#include <iostream>
using namespace std;

void asterisks(int num); // function prototype

int main()
{
    asterisks(5); // function call
}

void asterisks(int x)  // function definition
{
    for (int i=0; i<x; i++)
        cout << "*";
}
```

Example 3

Now an example of a function with two parameters.

```
#include <iostream>
using namespace std;

int sum(int num1, int num2);   // function prototype

int main ()
{
    int s, a, b;
    a = 4; b = 7;
    s = sum(a, b);             // function call
    cout << a << " + " << b << " = " << s;

    return 0;
}

int sum(int num1, int num2)    // function definition
{
    int sum;
    sum = num1 + num2;
    return sum;
}
```

Example 4

Function is_even checks whether the number provided to it is even or odd and returns 0 or 1 for even or odd number respectively. Third line in the program is the prototype declaration of the function is_even used in the program. As mentioned earlier compiler needs to know at least the name of the function before calling it. Method to give name of the function is to give the prototype of function as written in the below program above the main() function.

Note: *You cannot define a function inside another function, you can only call a function from inside another function.*

```
#include <iostream>
using namespace std;

int is_even(int n);   // function prototype

int main()
{
    int number, test;

    cout << "Enter a number to test whether it is even or odd" << endl;
    cin >> number;

    test = is_even(number);  // function call

    if (test == 0)
        cout << "The number is odd" << endl;
    else
        cout << "The number is even" << endl;

    return 0;
}

int is_even(int num)    // function definition
{
    int remainder;
    remainder = num % 2;

    if (remainder == 0)
        return 1;
    else
        return 0;
}
```

Tasks

Task 1:

Write a function called circarea() that finds the area of a circle. It should take an argument of type float and return a value of the same type. Write a main() program that gets a radius value from the user as input, calls circarea(), and displays the result.

Task 2:

Write a function largerDistance() that takes two distance values as arguments and returns the larger one. Write a main() program that takes two distance values, calls the largerDistance() function to find out the larger value, and prints the larger value.

Task 3:

Write a function absolute() that takes an int value as an argument and returns its absolute value. Create a main() program that inputs an int value, calls the function absolute(), and displays the result.

Hint: int abs(int n); is a pre-defined in the standard library <cstdlib>